

Chapter 56: Design a Rate Limiter

Personal Notes

The previous five chapters were sprawling designs — URL shortener, feed, chat, Uber, YouTube — each covering dozens of interacting services. This chapter is different: rate limiting is a FOCUSED problem. It's one component, usually middleware, that every real system needs. But the seemingly simple question "limit each user to 100 requests per minute" hides genuinely deep algorithmic and coordination challenges.

This chapter drills into five rate limiting algorithms (fixed window, sliding window log, sliding window counter, token bucket, leaky bucket), compares them precisely, then tackles the hard part: making rate limiting work across a distributed system with hundreds of servers. Along the way we'll write Redis Lua scripts for atomicity, handle race conditions, and design tiered limits for real APIs like Stripe or Twitter. Master this and you've mastered the core algorithm behind every API you'll ever build or consume.

1. The Problem

Design a rate limiter that caps how many requests a client can make in a time window. Prevent abuse (a bot hammering your login endpoint), protect against DDoS, keep costs predictable, and prevent one misbehaving client from degrading the experience for everyone else.

Client sends request → Rate Limiter checks:

- Who is this client? (user_id, API key, or IP)
- What endpoint are they hitting? (limits may differ)
- How many requests have they made in the last N seconds?

If under the limit:

ALLOW → forward request to service
Update counter

If at/over the limit:

REJECT → return 429 Too Many Requests
Include Retry-After header so client knows when to retry

Real examples:

Twitter API: 300 tweets / 3 hours per user
Stripe API: 100 read requests / second per API key
Google Maps: 50 requests / second per project

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Limit requests per client (user/IP/API key) over a time window
- Return appropriate error (429) when limit exceeded
- Support different limits per endpoint (login vs read vs write)
- Support different tiers (free user: 100/hr, paid: 10K/hr)
- (Optional) Global limits (total system capacity)
- (Optional) Dynamic limits that adjust based on backend load

2.2 Non-Functional Requirements

- Low latency — rate limiter adds < 5 ms to every request (it's in the hot path)
- High throughput — handle whatever QPS your services handle
- Fault tolerance — if the rate limiter goes down, DON'T take down the whole service
- Accuracy — being off by a few requests is usually fine; being off by 10× is not
- Distributed — must work across many API servers

2.3 Where Does It Sit?

- At the LB / API Gateway — first line of defense, blocks bad traffic early
- At each service — finer-grained per-endpoint limits
- At the client-side (SDK) — bandwidth/battery friendly, but easily bypassed

The critical clarification: "Is this the ONLY rate limiter, or is it one of several layers?" Modern systems have MULTIPLE — the LB does coarse IP-based limiting, the API gateway does per-key limiting, individual services add endpoint-specific limits. Each layer has different requirements. Confirm the scope upfront.

3. Step 2 — Capacity Estimation

Rate Limiter Throughput

Assume the service handles 1M QPS peak globally.
Rate limiter must handle EVERY request → 1M rate-check ops/sec.

Each check is a small counter operation:

- Look up counter for (client_id, endpoint)
- Compare against limit
- Increment if allowed

Latency budget: < 5 ms per check
(this is in the request hot path; every ms counts)

Storage

Data per client:
Counter, timestamp of last request, TTL info
~50 bytes per (client, endpoint) combination

100M active clients × 10 rate-limited endpoints × 50 B
≈ 50 GB in-memory

Fits in a small Redis cluster (5-10 nodes). Not a scaling problem.

4. Rate Limiting Algorithms

Five main algorithms, each with distinct trade-offs. The most-asked interview algorithm is TOKEN BUCKET; the most nuanced is SLIDING WINDOW COUNTER. You should know all five.

5. Algorithm 1 — Fixed Window Counter

Simplest possible. Split time into fixed windows (e.g., every minute). Count requests per window. Reject once the count exceeds the limit; reset at window boundary.

```
// Limit: 100 requests / minute per user

int count(String userId, long timestamp) {
    long windowStart = timestamp / 60 * 60;    // truncate to minute
    String key = userId + ":" + windowStart;
    int current = redis.incr(key);
    if (current == 1) redis.expire(key, 60);    // set TTL on first request
    return current;
}

boolean allow(String userId, long timestamp) {
    return count(userId, timestamp) <= 100;
}
```

Pros

- Trivially simple — a single INCR in Redis
- Very low memory (one counter per client per window)
- Fast — single atomic op

Cons — The Boundary Problem

Limit: 100 req/minute

Time 12:00:59 — client makes 100 requests (all allowed, window 12:00-12:01)

Time 12:01:00 — new window starts, counter resets

Time 12:01:01 – client makes another 100 requests (all allowed)

In a 2-second span, the client sent 200 requests – 2× the intended limit.

For attack traffic that happens to align with window boundaries, the actual limit is DOUBLE the configured limit. Not acceptable.

The boundary problem is why fixed window is rarely used alone: It's the naive approach that everyone reaches for first. In interviews, mentioning it and explaining WHY it's insufficient — then moving to a better algorithm — signals you've seen this pattern before.

6. Algorithm 2 — Sliding Window Log

Fix the boundary problem by tracking the **TIMESTAMP** of every request. Reject if the number of timestamps in the last N seconds exceeds the limit.

```
// Limit: 100 requests / 60-second sliding window

boolean allow(String userId, long timestamp) {
    String key = "log:" + userId;

    // Remove timestamps older than 60 seconds
    redis.zremrangebyscore(key, 0, timestamp - 60);

    // Count remaining timestamps
    long count = redis.zcard(key);

    if (count >= 100) return false;

    // Add current timestamp
    redis.zadd(key, timestamp, timestamp);
    redis.expire(key, 60);
    return true;
}
```

Pros

- Perfectly accurate — exactly the semantics "N requests in past 60 sec"
- No boundary issues — the window is continuously moving

Cons

- Memory-heavy — must store every timestamp within the window
- At 100 req/min per user × 100M users = 10 billion timestamps in memory
- For high-traffic clients (say 10K req/min), it's proportionally worse

7. Algorithm 3 — Sliding Window Counter (The Best Compromise)

Combines fixed window's efficiency with sliding window's accuracy. Track TWO counters (current + previous window) and estimate the sliding count via weighted overlap.

```
// Limit: 100 requests / 60-second window

boolean allow(String userId, long now) {
    long currWindow = now / 60 * 60;
    long prevWindow = currWindow - 60;

    long currCount = redis.get("cnt:" + userId + ":" + currWindow);
    long prevCount = redis.get("cnt:" + userId + ":" + prevWindow);

    // What fraction of the previous window still overlaps our sliding
    window?
    double elapsedInCurr = (now - currWindow) / 60.0;
    double weightPrev = 1.0 - elapsedInCurr;

    double estimatedCount = prevCount * weightPrev + currCount;

    if (estimatedCount >= 100) return false;

    redis.incr("cnt:" + userId + ":" + currWindow);
    return true;
}
```

How the Weighted Overlap Works

Windows (each 60 seconds):
[— prev —][— curr —]
 ▲
 |
 now (say, 25% into curr window)

Sliding 60-sec window = last 60 sec ending at now
Includes: 75% of prev window + 25% of curr window (so far)

Estimated count = $\text{prev_count} \times 0.75 + \text{curr_count} \times 1.0$

If prev had 80 requests, curr has 30:
estimate = $80 \times 0.75 + 30 = 60 + 30 = 90$
Under limit (100) → allow

Trade-off

- Accuracy: assumes uniform distribution within prev window — approximately right
- Memory: only 2 counters per client — same efficiency as fixed window
- Speed: 2 reads + 1 write per check — still very fast

Google's approach: Google uses sliding window counter (with variations) in most of their internal rate limiters. It's the pragmatic sweet spot — memory-efficient like fixed window, but without the boundary problem. When interviewers push you past "which algorithm would you use," this is the strongest default answer.

8. Algorithm 4 — Token Bucket

The most famous rate-limiting algorithm. Imagine each user has a BUCKET holding N tokens. Every request consumes 1 token. Tokens REFILL at a steady rate. Bucket has a max capacity. If empty, requests are rejected.

Bucket for user U:

Capacity: 100 tokens (max the bucket can hold)

Refill rate: 10 tokens / second

Current: 50 tokens

Request arrives:

- Compute tokens added since last check:
 $\text{tokens_added} = (\text{now} - \text{last_refill}) \times \text{refill_rate}$
- Cap at capacity: $\min(\text{bucket} + \text{tokens_added}, \text{capacity})$
- If bucket ≥ 1 : consume 1, allow request
- Else: reject, return 429

Example:

t=0: 100 tokens (full)

t=0.5: 50 requests → 50 tokens left

t=6: refill (6 sec × 10 tokens/sec = 60 added), capped at 100 = 100 tokens

Bucket is full again. Bursts allowed up to bucket capacity.

```
boolean allow(String userId, long now) {
    // Load current state
    Bucket b = redis.hgetall("bucket:" + userId);
    long tokens = b.tokens;
    long lastRefill = b.lastRefill;

    // Refill
    long elapsed = now - lastRefill;
    tokens = Math.min(CAPACITY, tokens + elapsed * REFILL_RATE);

    if (tokens < 1) {
        redis.hset("bucket:" + userId, "tokens", tokens, "lastRefill",
now);
        return false;
    }

    tokens -= 1;
    redis.hset("bucket:" + userId, "tokens", tokens, "lastRefill", now);
}
```

```
} return true;
```

Why Token Bucket Is Beloved

- Allows BURSTS naturally — full bucket = you can send 100 requests instantly if quiet before
- Steady average rate over time = refill rate
- Simple to reason about — real-world metaphor works
- Small memory — 2 values per client (tokens + timestamp)
- Amazon, Stripe, most cloud APIs use this pattern

9. Algorithm 5 — Leaky Bucket

Similar physical metaphor, different mechanics. Requests fill a bucket. The bucket "leaks" at a fixed rate — that's the throughput. If bucket overflows, requests are rejected.

Bucket for user U:
Capacity: 100 requests (max queue depth)
Leak rate: 10 requests / second

Requests come in → get added to bucket (queue)
Bucket drains at 10 req/sec regardless of arrival rate

If bucket is full → new requests are rejected

Key difference from Token Bucket:
Token bucket allows BURSTS then throttles
Leaky bucket SMOOTHS everything to constant rate

Token Bucket vs Leaky Bucket

Aspect	Token Bucket	Leaky Bucket
Bursts allowed?	Yes (up to bucket capacity)	No — smoothed to leak rate
Output rate	Variable (bursty)	Constant
Best for	APIs where occasional bursts OK	Traffic shaping, network QoS
Common use	Cloud APIs, user-facing services	Network routers, ISP throttling

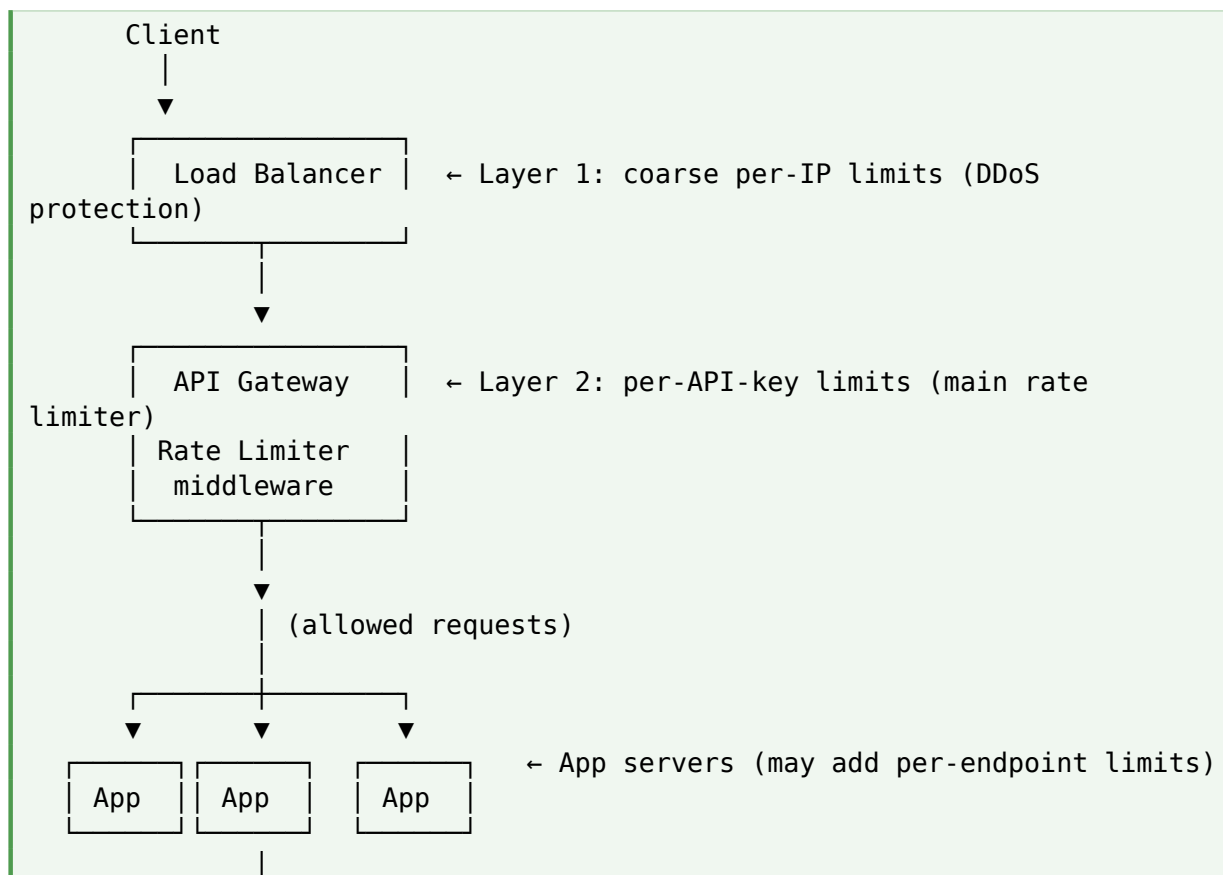
10. Algorithm Comparison

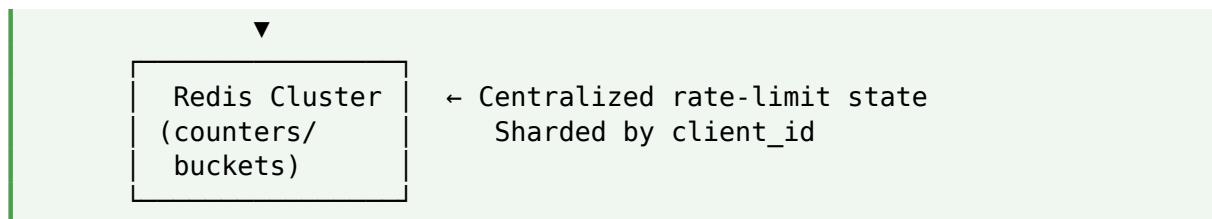
Algorithm	Accuracy	Memory	Bursts?	Best for
Fixed Window	Poor (boundary	Very low	Yes	Simple internal

Algorithm	Accuracy	Memory	Bursts?	Best for
	problem)			caps
Sliding Window Log	Perfect	High	No	Small, exact-limit systems
Sliding Window Counter	Good (approx)	Very low	No	Google-style API rate limits
Token Bucket	Perfect	Very low	Yes	Most cloud APIs
Leaky Bucket	Perfect (smoothed)	Depends	No	Traffic shaping

Interview default: Token Bucket: If asked "which algorithm would you use," default to Token Bucket. It's the industry standard, allows bursts (which real users need), has excellent memory profile, and is easy to explain. Mention Sliding Window Counter as an alternative if the interviewer wants a smoother rate.

11. Step 3 — High-Level Architecture





Layers

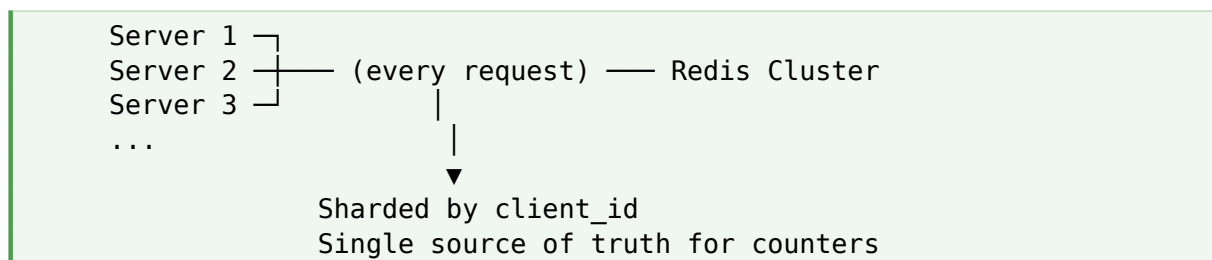
- LB — very coarse (per-IP, e.g., "1000 req/sec per IP") — protects infrastructure
- API Gateway — main per-key/per-user limits — protects application
- Service — per-endpoint fine-grained (e.g., "/login: 5 req/min per IP") — protects specific ops

12. Deep Dive: Distributed Rate Limiting

In production, you don't have ONE rate limiter — you have hundreds of API servers each seeing traffic. If each server enforces the limit independently, a user with limit "100 req/min" could send 100 to each server = $100 \times \text{server_count}$. Coordination is required.

Approach 1 — Centralized (Redis)

All servers query a shared Redis cluster. Redis holds the canonical counter/bucket state. Simple, accurate, but Redis is a critical dependency.



- Pros: perfect accuracy, simple to reason about, works with any algorithm
- Cons: Redis is now in the hot path — down Redis = broken rate limiter
- Latency: adds 1-3 ms per request (Redis round trip)

Approach 2 — Local + Sync ("Sloppy" Rate Limiting)

Each server keeps LOCAL counters. Periodically sync with peers (via a shared store) to reconcile. Never perfectly accurate — but very fast.

- Pros: near-zero latency (in-process), works even if central store is down
- Cons: allows overshoot proportional to $(\text{peers} \times \text{sync_interval})$
- Best when: strict limits aren't critical (e.g., cost control, not security)

Approach 3 — Sticky Routing

LB routes all traffic from the same client to the SAME server. That server owns the client's counter, no coordination needed. Simple but couples LB and rate limit design tightly.

- Pros: no distributed coordination, super fast
- Cons: hot server if one client has extreme traffic; server crash loses state briefly

The senior answer: "For a global API like Twitter, I'd use centralized Redis-based rate limiting with sliding window counter or token bucket. The 1-2 ms latency overhead is acceptable given the perfect accuracy. For internal, latency-critical services, I'd use sloppy local counters with periodic sync — accepting ~5% overshoot in exchange for zero coordination overhead."

13. Deep Dive: The Race Condition

Two requests arrive at the same instant. Both read the counter as 99 (under limit 100). Both increment. Now the counter is 101 — but 2 requests went through when only 1 should have. This is the classic READ-MODIFY-WRITE race.

The Fix — Atomicity

- Redis INCR — atomic increment, single command. Perfect for fixed window.
- Redis Lua Script — for multi-step operations (token bucket refill + consume), run as a single atomic block.
- Redis MULTI / EXEC — transaction; but doesn't handle read-decide-write well.

Token Bucket via Lua Script

```
-- Lua script executed atomically in Redis
-- KEYS[1] = bucket key
-- ARGV: capacity, refill_rate, now

local bucket = redis.call("hmget", KEYS[1], "tokens", "last_refill")
local tokens = tonumber(bucket[1]) or tonumber(ARGV[1]) -- default: full
local last = tonumber(bucket[2]) or tonumber(ARGV[3])

local elapsed = tonumber(ARGV[3]) - last
local refilled = math.min(tonumber(ARGV[1]),
                           tokens + elapsed * tonumber(ARGV[2]))

if refilled < 1 then
    redis.call("hmset", KEYS[1], "tokens", refilled, "last_refill",
    ARGV[3])
    return 0 -- reject
end

redis.call("hmset", KEYS[1], "tokens", refilled - 1, "last_refill",
    ARGV[3])
redis.call("expire", KEYS[1], 3600)
return 1 -- allow
```

Never do read-decide-write across two network calls: Even if it "seems to work" in single-threaded testing, at scale you'll get race conditions. Redis Lua scripts run atomically in the server's single-threaded event loop — same for Cassandra/DynamoDB conditional writes. Non-negotiable pattern for rate limiters.

14. Deep Dive: Rate Limit Policies

Tiered Limits

```
Rate limit policy (JSON config):
{
  "user_tiers": {
    "free":    { "requests_per_min": 60,  "burst": 10 },
    "basic":   { "requests_per_min": 300, "burst": 30 },
    "pro":     { "requests_per_min": 3000, "burst": 300 },
    "enterprise": { "requests_per_min": 30000, "burst": 3000 }
  },
  "endpoint_multipliers": {
    "GET /search": 1.0,           // normal cost
    "POST /login": 10.0,          // expensive – count as 10 requests
    "POST /export": 100.0         // very expensive
  }
}
```

Endpoint-Specific Limits

- /login: strict (5 req/min per IP) — password brute-force protection
- /signup: strict — bot registration protection
- /search: moderate — expensive backend
- /read-user: loose — cheap, common

Allowlists / Denylists

- Allowlist: trusted IPs/keys skip rate limiting entirely (internal services, partners)
- Denylist: known-bad IPs get rejected before rate check (fast fail)
- Store in Redis + local cache; refresh every 5-10 minutes

15. Deep Dive: Response to Rate Limit

What do you send back when a client exceeds their limit? Getting this right matters — bad responses cause client retry storms.

HTTP Response

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
X-RateLimit-Limit: 100
```

```
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1699500000
Retry-After: 42

{
  "error": "rate_limit_exceeded",
  "message": "You have exceeded 100 requests/min. Retry after 42 seconds.",
  "retry_after": 42
}
```

Standard Headers

- X-RateLimit-Limit — total limit in this window
- X-RateLimit-Remaining — how many left in current window
- X-RateLimit-Reset — Unix timestamp when window resets
- Retry-After — seconds the client should wait before retrying

Retry-After is critical for well-behaved clients: Without a clear "wait N seconds" hint, misbehaved clients retry immediately in a tight loop — turning a rate limit response into a DDoS on your rate limiter itself. Good SDKs (Stripe, AWS) respect Retry-After automatically.

16. Additional Features

16.1 Dynamic Limits

Adjust limits based on backend load. When the DB is at 95% CPU, tighten limits to reduce load. When idle, relax them for better user experience. Requires a feedback loop: monitor backend metrics → update limit config in Redis → next requests use new limits.

16.2 Distributed Consensus

For CRITICAL limits (billing quotas, security), consider stronger coordination: a lease-based system where a central service assigns capacity chunks to each API server. Doesn't have to be Raft-level — a periodic reconciliation is often enough.

16.3 Cost-Weighted Limiting

Different endpoints have different backend cost. Instead of "100 requests", limit by "100 cost units" where /search costs 10 units and /read costs 1. Prevents users from monopolizing expensive endpoints.

16.4 Client-Side Rate Limiting

SDK enforces the limit before even sending the request. Reduces network waste, saves battery on mobile, but easily bypassed by determined attackers. Complementary to server-side, never a replacement.

17. Failure Modes

Failure	Mitigation
Redis cluster down	FAIL-OPEN — allow requests through rather than blocking legitimate traffic. Log alerts.
Redis latency spike	Local cache with short TTL as fallback. Timeout after 5 ms → allow.
Config service down	Continue with last-known-good limits. Never block on config fetch.
Bug in limiter logic	Kill switch — feature flag that disables rate limiting entirely.
Sudden legitimate spike	Auto-scale backend + dynamic limits to accommodate. Better than 429 storm.
Attacker rotates IPs	Additional signals: user-agent fingerprint, TLS fingerprint, behavior.

Fail-open vs fail-closed: Rate limiters typically FAIL-OPEN — if the limiter is broken, allow traffic rather than block. Blocking legitimate users because your rate limiter is down is worse than briefly allowing over-limit traffic. Contrast with auth: auth fails CLOSED (deny by default). Understanding this distinction is a senior-level insight.

18. Trade-offs Summary

Decision	Trade-off
Fixed window vs sliding	Fixed = simplest, boundary problem; sliding = accurate, more complex
Token bucket vs leaky bucket	Token = allows bursts; leaky = smooths to constant rate
Centralized (Redis) vs local	Centralized = accurate + coordination overhead; local = fast + sloppy
Fail-open vs fail-closed	Fail-open = availability; fail-closed = strictness (rare for rate limiting)
Client-side vs server-side	Client = bandwidth-friendly + bypassable; server = source of truth
Sticky routing vs distributed	Sticky = simple + hot-key risk; distributed = balanced + coordination
Static vs dynamic limits	Static = predictable; dynamic = adapts to load, complex to tune
Per-user vs per-endpoint	Per-user = fair; per-endpoint = protects specific ops; often use both

19. Common Mistakes

Mistake 1: Using fixed window without addressing the boundary problem

Fixed window is fine for many cases, but always MENTION the 2× overshoot possibility. Interviewers watch for this awareness — it separates candidates who understand the algorithm from those who just memorized it.

Mistake 2: Read-then-write without atomicity

GET count from Redis, check limit, if OK do INCR. This has a race. Always use INCR (atomic) or a Lua script for token bucket. Even for a whiteboard interview, mention this explicitly.

Mistake 3: Fail-closed by default

If Redis is down, do you block ALL traffic? For most APIs, no — that's worse than allowing over-limit traffic briefly. Fail-open by default; escalate + alert.

Mistake 4: One rate limit fits all

Free vs paid users, cheap vs expensive endpoints, humans vs bots — they need different limits. Design tiered/endpoint-specific limits from the start.

Mistake 5: No Retry-After header

Return 429 without telling the client when to retry → the client retries in 100 ms → hits limit again → retries again. You've created a retry storm on your own limiter. Always include Retry-After.

Mistake 6: Ignoring the distributed coordination question

If your design has multiple servers, HOW does the count stay consistent? Centralized Redis, local + sync, or sticky routing? Interviewers will ask — have an answer.

Mistake 7: Rate limiting only on IP

Behind a corporate NAT or CGNAT, thousands of legitimate users share one IP. Pure IP-based limits punish them all. Combine IP with user_id, API key, or session for fairness.

20. Best Practices

- Default to TOKEN BUCKET for user-facing APIs (allows natural bursts)
- Use SLIDING WINDOW COUNTER for smooth-rate systems and analytics
- Store counters in centralized Redis; use Lua scripts for atomic multi-step ops
- FAIL-OPEN — if the limiter breaks, allow traffic and alert
- Always return Retry-After header on 429 responses
- Layer rate limiting: LB (IP) → Gateway (key) → service (endpoint)
- Design tiered limits from day one (free / paid / enterprise)
- Combine IP + user_id to avoid punishing legitimate NAT users

- Include kill switch — feature flag to disable rate limiting in emergencies
- Monitor rate-limit-triggered 429s per user/endpoint — alerts on anomalies

21. Quick Reference

Concept	Meaning
Rate limiter	Caps requests per client per time window
Fixed window	Count per fixed interval; boundary problem
Sliding window log	Track every timestamp; accurate but memory-heavy
Sliding window counter	Weighted overlap of two windows; accurate + cheap
Token bucket	Tokens refill at rate; consume 1 per request; allows bursts
Leaky bucket	Requests fill bucket; drains at fixed rate; smooths bursts
Fail-open	If limiter breaks, allow traffic (opposite of auth)
429 Too Many Requests	HTTP status for rate limit exceeded
Retry-After	Header telling client when to retry
Lua script	Atomic multi-step operation in Redis
Centralized limiting	All servers share Redis counter — accurate, ~2ms overhead
Sloppy limiting	Local counters + periodic sync — fast, some overshoot
Kill switch	Feature flag to disable rate limiting instantly

22. Related Interview Problems

Rate limiter patterns extend to many related systems:

1. Design a Quota System — count "units consumed" (API calls, GB storage, compute hours) against a monthly limit
2. Design a Distributed Semaphore — cap concurrent operations, not just rate
3. Design a Circuit Breaker — pattern for cascading failure prevention
4. Design a Fair Queuing Scheduler — balance many clients' request throughput
5. Design a Traffic Shaper for a CDN — smooth burst traffic to origin
6. Design a Login Throttler — brute-force protection with exponential backoff
7. Design a Feature Flag System — control rollout percentage per user
8. Design a Distributed Counter — closely related; e.g., view count aggregation

23. Final Takeaway

Rate limiter is one of those problems where the algorithm choice matters more than the architecture. Get the algorithm right (Token Bucket, Sliding Window Counter) and the rest is standard distributed systems (Redis for state, Lua for atomicity, layers for defense in depth). Get it wrong (naive fixed window, race-prone read-then-write) and the system fails silently in exactly the scenarios you built it for — bots and attacks.

For interviews, memorize: (1) the 5 algorithms and their trade-offs, (2) Token Bucket is your default answer, (3) centralized Redis + Lua for atomicity, (4) fail-open by default, (5) always include Retry-After. Then be ready for the deep-dive: "How do you rate-limit across 100 servers? What happens when Redis is down? What if a user rotates through 10 million IPs?" These are the questions that separate good candidates from great ones.

Key Takeaway: Rate limiter = classic distributed coordination problem. FIVE algorithms: FIXED WINDOW (simple, boundary problem), SLIDING WINDOW LOG (accurate, memory-heavy), SLIDING WINDOW COUNTER (best compromise), TOKEN BUCKET (industry default, allows bursts), LEAKY BUCKET (smooths to constant rate). Store state in CENTRALIZED REDIS with LUA SCRIPTS for atomicity. FAIL-OPEN by default. Always return Retry-After header. Layer defenses: LB (IP) → Gateway (key) → service (endpoint). Design tiered limits and kill switches from day one.

24. What's Next?

Now that rate limiting is covered, remaining directions:

- Design a Distributed Cache — Redis/Memcached internals, consistent hashing, replication
- Design a Notification System — cross-channel delivery, priorities, batching
- Design a Search Autocomplete — trie-based, prefix caching, popularity scoring
- Design a Payment System — idempotency, sagas, PCI compliance
- Design a Web Crawler — distributed fetching, politeness, dedup
- Design a Metrics / Monitoring System (like Datadog)
- Design a Distributed Job Scheduler
- Deep Dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once
- Deep Dive: Consensus Algorithms — Raft, Paxos, leader election